

Bootstrapping historical transaction replay for SVMscope

Recommendation. Build the missing historical state into SVMscope's own archive using public checkpoints, historical transactions and deterministic reconstruction. Start with the verified Whirlpool dataset, add narrowly validated state reconstruction for other programs, and use canonical ledger execution to complete coverage where those shortcuts cannot establish every required input. This removes the calendar wait for accounts whose history can be imported or reconstructed. It does not make an incomplete dataset complete merely by giving it a 30-day date range.

The most significant finding is a publicly downloadable August 11, 2026 Whirlpool checkpoint containing **1,052,402 account-data entries and the historical program binary**. A complete download passed gzip integrity checking. The checkpoint includes the pool and three tick arrays required by the first sampled August 12 swap. Historical concentrated-liquidity state therefore has a concrete, free source for this protocol; the earlier blanket claim that it requires a paid account-state API was incorrect.^{[1][2][3]}

The target remains **an arbitrary transaction evaluated against state at an arbitrary historical slot**, rather than reproducing only its original execution. The implementation must resolve all inputs at the target position. A transaction's own balance metadata describes its original position and cannot supply balances at some other date. The universal guarantee is an acceptance condition for the completed system; it has not been demonstrated by the available prototypes.

Scope and dates. The requested August 12–September 13 span exceeds 30 days. Preserve that larger bootstrap range, then enforce the rolling 30-day product window using actual finalized chain time. September 13 is the local date in India; the observations below were made on September 12 UTC. Future slots are outside the initial import. At the sampled finalized tip, the chain was at slot 446,554,585 in epoch 1033. The independently inspected start checkpoint was slot 438,704,931, timestamped August 11 at 23:59:59 UTC. Its next instruction file begins at slot 438,704,932, August 12 at 00:00:00 UTC.^{[3][4]}

1. The cold-start design

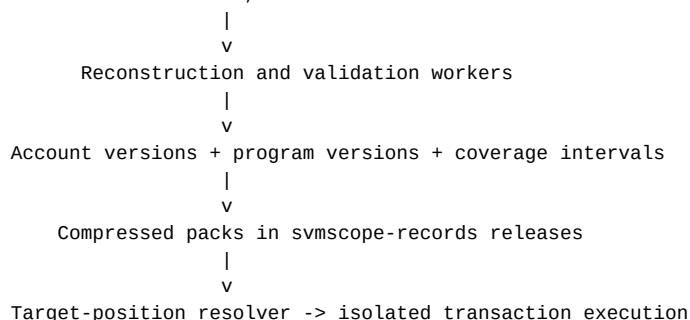
The useful archive object is a versioned account state with a defensible validity interval. Transactions are the transition inputs used to construct those versions. For a known anchor A and target T, the computation is: load state at A, apply every relevant canonical transition through T, then execute the requested transaction in an isolated copy. Account creation, deletion and changes of ownership are transitions too. A complete anchor plus complete execution inputs gives a general reconstruction route; a protocol checkpoint can establish a much smaller starting state for a supported subset.

Solana transaction responses expose instructions, account addresses, selected balances, execution logs and related metadata. They do not normally include arbitrary pre- and post-execution bytes for every account. The account model separately includes data, lamports, owner, executable status and rent metadata. Consequently, dumping transaction JSON is useful raw material, but its usefulness depends on the reconstruction code and starting state attached to it.^{[5][6]}

Use a provider interface that resolves (address, chain position) from an imported checkpoint, a complete mutation journal, a validated program-specific reconstruction, or a canonical replay worker. Resolve dependencies recursively and cache the result. Persist resolved history so later requests reuse work. A provider may return complete account state, selected fields, verified absence, or unresolved state; those are different results and must remain distinguishable throughout the pipeline.

The proposed architecture is:

Public protocol checkpoints + ordered protocol instructions
Public ledger blocks + transaction metadata + deployment history
Verified full-state anchors, when obtained



This preserves the reason to build SVMscope: it owns the materialized history, indexing, reconstruction, execution and serving layer. Importing public seed data is normal database bootstrapping. It does not require routing every visitor's query through somebody else's paid account archive.

2. Verified Whirlpool input, including actual file contents

Orca publishes the `whirlpool-tx-replayer` repository and links its public Pleiades archive. The inspected implementation supports daily checkpoint loading, historical program changes, ordered instructions and callbacks around execution. Its schema stores account addresses and base64 data, with a separate program binary. These are useful historical bytes, but the schema is narrower than a complete Solana account record.^{[1][7][8]}

The August 11 checkpoint is 125,637,979 compressed bytes. It contains 1,584,223,548 bytes of decoded account data and a 1,537,784-byte ELF program. Its SHA-256 is 5d2a4ac12ba98c764f9f61f1713494f8858d573cd06518c54e2a9bb6a4277cc9. Streamed inspection took approximately 20 seconds after download; this measures parsing and integrity checking, **not transaction execution speed**.^[3]

The first sampled transaction in the August 12 file is at transaction index 415 in slot 438,704,932. Its Whirlpool account is `HktfL7iwGKT5QHjywQkcDnZXScoh811k7akrMZJkCcEF`. The checkpoint contains that account's 653 data bytes and all three referenced 9,988-byte tick arrays. The referenced oracle address was not among the matched entries; its historical existence and the applicable instruction behavior require separate handling. Per-account hashes and the sample signature are preserved in the evidence JSON.^[3]

The source index was updated at September 12, 12:46:03 UTC and listed daily files through September 10. All requested checkpoint dates from August 11 through September 10 were indexed. September 11 onward was not listed at that observation. Only one checkpoint was downloaded in full; the August 12 transaction file was inspected through a 1 MiB range request. Listing a file establishes advertised availability, not that every file has passed integrity and replay validation.^{[2][3]}

Import strategy	Compressed bytes	Purpose
August 11 checkpoint only	125,637,979	Initial Whirlpool data and program anchor
August 12–September 10 instruction files	12,770,117,250	Ordered protocol transitions
One anchor plus those instructions	12,895,755,229	Lowest-transfer sequential bootstrap
All August 11–September 10 anchors plus those instructions	16,787,834,010	Independent daily jobs and checkpoint comparisons

These totals describe Whirlpool inputs, not an all-Solana account archive. The supplied CSV contains exact URLs, sizes, roles and verification status. The all-anchor strategy costs about 3.9 GB more transfer and

enables independent daily reconstruction because each day has its own preceding checkpoint. That is a concrete form of parallelism unavailable when only one whole-chain anchor exists.^[2]

A crucial code-level limit prevents treating Orca's replayer as the finished SVMscope engine. The inspected swap adapter constructs token accounts with balances selected to make the protocol instruction executable. It also contains special handling for the oracle account's writability. Such arrangements can support reconstruction of Whirlpool-owned data, but they do not establish historical token-account bytes or fidelity of an entire original transaction. Use this engine as a candidate state producer, then validate its output and supply missing dependencies separately.^[9]

3. The first implementation: import and validate Whirlpool history

Pin the inspected upstream revision, 95b595cad50f1f5147c3524613c805c516f03005. Add an importer that reads a checkpoint without expanding the complete JSON to disk. Store account data and historical ELF hashes in a separate staging namespace. Tag imported records with the publisher, source URL, compressed-file hash, checkpoint slot and exact field coverage. At this stage, label them as publisher-supplied account data, not independently verified complete account state.

Run one full daily replay before starting a month-wide job. Load the preceding day's checkpoint, apply the day's supported instructions and export changed Whirlpool-owned bytes. Compare the resulting address set and bytes against the independently downloaded end-of-day checkpoint, including accounts created and closed. Reject the interval if an instruction is unsupported or a required transition is missing. A successful end-of-day comparison establishes agreement with the publisher's checkpoint; since both files share a publisher, it is not independent consensus verification.

For stronger validation, sample intermediate slots against a separate authoritative state source where available, and cross-check original transaction results using raw canonical blocks. Do not use price agreement alone as an account-state test. The published checkpoint and instruction sequence may contain correlated errors; sample checks and program-version verification reduce that risk but should not be described as a cryptographic proof of the entire chain.

After the one-day gate passes, dispatch one job per day using the preceding checkpoint. Capture updates in transaction order, preserving instruction order within a transaction while publishing durable account state at the transaction boundary. Produce block-end versions for arbitrary-slot requests. Keep checkpoint verification and export in the same job so an unverified file cannot accidentally be promoted into the serving manifest.

Bridge the recent tail from the September 10 checkpoint using canonical block or address-history retrieval. Extract all relevant instructions, including inner calls, and adapt them to the pinned decoder. Discover accounts created during the gap rather than restricting ingestion to today's watched list. Skip unsuccessful protocol state changes appropriately, while leaving whole-transaction fee and other runtime effects to the full-state layer. This bridge is real implementation work: neither the current polling recorder nor merely downloading September 10's checkpoint fills the missing days.

A useful first acceptance example is a transaction submitted against a target slot on a different day from its original execution. The resolver must select that target day's pool, ticks, token state, program versions and runtime context. Test targets before pool creation and after account closure as well. Historical execution failure due to absent funds or accounts can be the correct result; successful execution is not the definition of historical accuracy.

4. Public ledger access without downloading all transactions

Old Faithful publishes epoch CAR files together with indexes for slots, signatures, content offsets and address history. Its HTTP configuration supports range retrieval from remotely hosted CAR files. This gives two useful modes: indexed retrieval of selected blocks or transactions, and sequential streaming for broad backfills. Address indexes still have their own download and serving costs; selective retrieval is not zero-cost random access.^{[10][11][12]}

Read-only HEAD requests succeeded for every completed epoch 1015–1032 overlapping the requested window. Their combined advertised CAR size is **17,480,804,161,166 bytes**, or approximately **17.48 TB / 15.90 TiB**. This includes complete boundary epochs, rather than precisely the requested slot subset, and excludes indexes, account snapshots and generated state. The current epoch 1033 object returned HTTP 404, so a recent-block source must supply that tail.^[13]

Do not begin by mirroring these files into GitHub. For request-driven reconstruction, use address-history indexes to find relevant transitions and slot/signature indexes to fetch their records. Persist the selected raw inputs for reproducibility. For a broad protocol scan, stream blocks through a filter and write compact protocol records. For canonical whole-chain execution, preserve all runtime-relevant transactions and transitions rather than assuming a user-transaction-only export is sufficient.

Anza's Jetstreamer is an appropriate candidate for streaming and indexing historical ledger data. Its advertised multi-million-transaction throughput measures an ingestion workload; it is not a benchmark for re-executing those transactions and storing every account write. Superbank is another historical query/indexing component. Neither should be mistaken for an already populated historical account-byte database.^{[14][15]}

The practical order is protocol imports first, targeted ledger retrieval second, and a bulk scan only where measurements show it amortizes better across many requests. Track bytes fetched, index size, reconstruction depth and cache hit rate. A dependency graph can become large, but it does not automatically become the entire chain for every request. Stop expansion at a verified anchor, a known creation state or an independently sufficient reconstruction boundary.

5. Extend coverage through validated reconstruction

The existing Pump experiment is useful evidence that transaction history can recover more than the earlier discussion acknowledged. Three selected original-slot transactions from August 13, August 23 and September 4 reproduced their reported trade amounts and four reserve fields after historical reserve reconstruction. Their baselines either produced different amounts or failed. Across the sampled blocks, all 31 checked consecutive reserve transitions agreed.^[16]

That result is deliberately narrower than exact replay. It uses current program binaries and remaining account fields, and compute consumption differs from the chain in all three samples. It derives candidate pre-state using the evaluated transaction's own event, so matching the same event is not an independent historical-input test. The next test must reconstruct the target from earlier transitions, hold the evaluated transaction out, and then run it. The relevant Pump IDL supplies the event and instruction definitions; every supported layout and upgrade era needs explicit handling.^[17]

Build protocol adapters around complete mutation coverage. For a Pump curve, recover creation parameters, reserve changes, completion/migration status, applicable authority/configuration changes and the historically active binary. For token accounts, reconstruct initialization, owner and authority changes, delegate state, freezing, closing and supported extensions; a numeric balance alone is insufficient. For a concentrated-liquidity pool, include liquidity changes, ticks, fee/reward accounting and applicable oracle state. Add an adapter only when its input requirements and unsupported operations are explicit.

Historical program recovery is also a tractable subproblem in some cases. Upgradeable-loader instructions include writes with offsets and byte payloads, followed by deployment or upgrade operations referring to a

buffer. With complete buffer history, a worker can rebuild the deployed bytes even if the temporary buffer is now closed. This is a proposed reconstruction route grounded in the instruction format, not a completed SVMscope implementation. Missing write history, other loader versions or program activation semantics remain separate cases.^[18]

For each adapter, distinguish full account state from a set of reconstructed fields. A resolver may combine compatible field-level evidence only when it refers to the same chain position and account incarnation. Filling unknown bytes with today's values is an approximation. No adapter may upgrade that result to complete state simply because a sample transaction happens to succeed.

6. Completing universal coverage with canonical state replay

For arbitrary programs with opaque or incomplete reconstruction inputs, the general route is a verified complete bank/account anchor preceding the target, followed by canonical execution to the target. Export the resulting account mutations into the same serving format. Protocol imports accelerate this architecture; they do not replace the full-state requirement for transactions whose remaining dependencies cannot be resolved otherwise.

A complete August 11 whole-chain snapshot with verified anonymous download access has **not** been obtained. Documented warehouse buckets were investigated: the US and European list requests required a requester billing project, while the Asian request rejected anonymous listing. These are access observations, not evidence that historical snapshots do not exist. A raw snapshot transfer is a different dependency from subscribing to a per-query historical account API.^{[19][20]}

Google currently documents a promotional pricing exception for certain Storage Transfer Service access to requester-pays buckets. It still requires an eligible billing project, access permissions and consideration of destination storage costs. It is an acquisition lead for an existing cloud setup, not a verified zero-cost bootstrap. No billing project or paid transfer was activated for these observations.^[20]

Once a compatible snapshot is available, use a matching Agave replay environment and capture ordered writes through Geyser or a replay-specific export path. Geyser already provides account-update interfaces, and historical persistence plugins exist; the assertion that all capture plumbing must be invented was too strong. Snapshot extraction can be streamed through tools such as `solana-snapshot-etl`, subject to compatibility testing with the snapshot era.^{[21][22][23]}

Firedancer includes an offline backtest command, and Mithril explores a smaller full-node implementation. They are candidates for measured replay experiments, not evidence of a guaranteed completion time on this laptop. Mithril's current documentation identifies historical replay compatibility as unfinished work. The previous fixed claims about thousands of dollars, a mandatory 512 GB machine or replay always running near real time were not established by a benchmark of this job.^{[24][25]}

Whole-chain work can be divided into independent intervals only where compatible starting anchors exist. Within an interval, canonical execution still has dependencies even if the runtime schedules non-conflicting transactions concurrently. Record bank-hash or equivalent canonical validation results at checkpoints and preserve the exact runtime/build configuration. A matching full-state commitment is stronger evidence than comparing just swap outputs.

7. Other sources investigated

Source	Useful contribution	Status for this bootstrap
Orca/Pleiades	Historical Whirlpool bytes, ELF and instruction files	Concrete input; checkpoint downloaded and inspected
Old Faithful	Canonical historical ledger and indexes	Required completed epoch objects accessible; current-epoch tail separate
Public Solana RPC	Selected historical transactions and recent blocks	Used by local probes; throughput and completeness must be measured
Solarchive / Hugging Face	Public analytical account and transaction datasets	Inspected index dated December 2025; account directories end in December 2025
Cloudbreak	Snapshot ingestion and live account read infrastructure	Useful building block; does not itself supply the missing August history
SVS Rewind	Advertised historical state and simulation service	External service path; required-date access and export coverage unverified
k256 Replay	Advertised historical mainnet frames and local execution	Frame catalog is a lead; no required-date downloadable seed was verified
datastore.sh	Protocol-specific downloadable analytical tables	Catalog inspected; complete raw-state coverage and free access unverified

Solarchive merits a precise correction: its website advertises a broad archive, but the inspected public index and account-directory listing do not establish August 2026 coverage. Its linked website schema endpoint returned 404; a Hugging Face schema directory exists. Therefore, the dataset cannot be selected as the required seed on the strength of its landing page alone.^{[26][27]}

Cloudbreak, SVS, k256 and datastore.sh remain useful references for architecture or future source expansion. They are not dependencies in the recommended initial free import. No claim of a turnkey, anonymously accessible full-Solana state dump for every requested slot is supported by the checked sources.^{[28][29][30][31]}

8. Store the result in the existing GitHub archive

The archive repository is public. Its present code already uses GitHub Release assets, keeping data outside Git history. Preserve that approach. GitHub documents a limit of 1,000 assets per release and requires each asset to be smaller than 2 GiB; it does not document a total release-size or bandwidth quota on that page. Accordingly, a multi-gigabyte bootstrap is not ruled out by the per-file limit.^{[32][33]}

Publish compressed packs partitioned by date and program or address range, with an implementation target of 256 MiB per pack. That size is a proposed operational choice, not a measured optimum. Each manifest should include schema version, covered positions, full versus partial fields, source hashes, program hashes, compression, checksum and a gap list. Publish the manifest only after every referenced pack has passed validation and upload verification.

Retain the predecessor state needed to answer the oldest supported query. An account last written 45 days ago may still supply its exact state throughout the newest 30 days. Pruning by file date alone would destroy that anchor. Keep a rolling boundary checkpoint plus all subsequent required changes, and retain deletion tombstones and account incarnation identifiers. A quiet account does not need one identical copy per slot if a complete update stream proves that it was unchanged.

Standard public GitHub Actions runners are a plausible compute venue for daily protocol jobs: GitHub currently documents free standard execution for public repositories, with 4 CPU cores, 16 GB RAM and 14 GB SSD for the public Linux x64 runner. Hosted jobs have a six-hour execution limit. These specifications suggest a bounded day-at-a-time experiment; they do not establish that the pinned replayer and its build artifacts fit until tested.^{[34][35]}

Build once, reuse a pinned executable/container where practical, and process compressed input as a stream. Keep temporary working state below a measured disk ceiling and upload each completed pack before releasing temporary files. Begin with low concurrency and increase only after checking publisher access behavior, job resources and aggregate download throughput. Use Releases for durable served packs rather than assuming Actions artifact storage is interchangeable with release storage.

9. Changes required in this repository

These are proposed implementation work packages. They are not represented as completed production changes.

Work package	Repository location	Completion criterion
Historical position and provenance types	<code>src/records.rs</code> , <code>src/scope.rs</code>	Distinguish slot boundaries, transaction positions, absent accounts and partial fields
Whirlpool checkpoint/import worker	New <code>src/history/orca.rs</code> and import example	One full day reconstructs and matches its published ending checkpoint
Recent-tail decoder and ingestion	New <code>src/history/ledger.rs</code>	September 10 checkpoint connects continuously to finalized live coverage
Historical program and dependency resolver	New <code>src/history/programs.rs</code> , <code>src/scope.rs</code>	Target-era binaries and all required account fields resolved without current-state substitution
Versioned packs and safe retention	<code>src/records_github.rs</code>	Restart restores the same coverage; oldest-query anchors survive pruning
Exact execution and comparison	<code>src/replay.rs</code> , <code>examples/fidelity_sweep.rs</code>	Original-position and different-target tests pass their separate contracts
Bootstrap automation	New bounded import workflow	Resource measurements recorded; interrupted jobs resume without duplicate promotion

Two existing storage behaviors directly affect the cold start. `src/records.rs` uses confirmed polling and a dense tier of 216,000 slots, then retains only the newest version per 75-slot bucket; `covers()` also rejects sufficiently old dense coverage. Those rules cannot preserve arbitrary-slot exactness for the entire imported month. Add a separate complete-history retention mode before importing valuable historical versions, otherwise compaction can discard the information the bootstrap just recovered.^[36]

Coverage also needs to be per account and interval, supported by complete transitions or verified anchors. A global recorder start time or the fact that an address was watched does not prove that every intermediate write was captured. Use finalized canonical account updates, or reconstruct complete ordered mutations from a validated source. Keep polling observations available as observations, while the exact historical provider uses the stronger coverage contract.

Define two execution contracts. **Original execution** reconstructs state immediately before the transaction, including relevant earlier writes in its block. **Execution at another slot** defaults to the finalized end of that produced slot, then evaluates the transaction in isolation. If the product substitutes a blockhash, bypasses signature checks or overrides balances, report those simulation changes explicitly. Do not compare a transaction inserted at a different target against its original outcome as though equality were expected.

For original execution, slot S minus one alone can miss earlier writes in slot S. For another target, original transaction balances are the wrong baseline. Program activation, relevant sysvars, feature configuration and address lookup resolution must match the chosen contract. These requirements belong in the resolver and execution boundary; they are not solved by adding more seed addresses.

10. Delivery sequence and measurable time estimates

Start with the artifact already obtained: the August 11 Whirlpool checkpoint and the file manifest. The first engineering milestone is one validated historical day, not a month-wide download. That milestone yields actual import speed, execution speed, output size and memory use. It also exposes unsupported instructions before compute is multiplied across thirty jobs.

Then run independently anchored daily jobs and the recent-tail bridge, assemble token/program dependencies, and execute a held-out different-target transaction. Add protocol adapters in order of observed unresolved dependencies, using cache statistics from real requests. In the same architecture, the canonical replay worker supplies general account coverage once its starting snapshot and runtime are available. This keeps the universal objective intact while avoiding unnecessary whole-chain work for requests already fully resolved by smaller inputs.

The following calculations are transfer-only lower bounds using advertised file sizes. They exclude execution, decompression, index transfer, rate limiting, uploads and contention. They are useful for selecting a strategy, not for promising a launch time.

Input set	At a sustained 100 Mbit/s	At a sustained 1 Gbit/s
12.896 GB Whirlpool anchor + instructions	17.2 minutes	1.72 minutes
16.788 GB Whirlpool daily anchors + instructions	22.4 minutes	2.24 minutes
17.481 TB completed whole-ledger epoch files	388.5 hours	38.85 hours

Estimate the actual protocol-job duration as download time plus instruction count divided by measured execution throughput, plus state export and validation time. With independent daily anchors, completion is bounded by the slowest batch under the chosen concurrency, not thirty sequential days. For whole-chain replay, measure slots or transactions executed per second and account bytes exported per second on the actual runtime and storage device. Do not use a historical ingestion benchmark as the denominator.

A launch gate should require: a declared target-position contract; complete inputs for the requested transaction; no unresolved source gaps across its needed history; pinned historical programs and required runtime context; deterministic reruns; and independent validation appropriate to the exactness claim. Test multiple writes in one slot, quiet accounts, account creation and closure, upgrades, failed transactions and target dates different from the original. A network-wide guarantee additionally requires coverage for arbitrary programs and dependencies, not just successful sampled swaps.

Decision. Implement the public-checkpoint import and state resolver first. There is a concrete path to historical coverage that starts before the recorder existed, and the verified Whirlpool source substantially reduces its initial workload. The outstanding universal-coverage dependency is complete historical state and execution context for everything outside validated imports and reconstruction. The next defensible milestone is a full-day reconstruction plus a complete different-target transaction replay; a same-day, network-wide guarantee remains unverified until those broader coverage gates are satisfied.

Evidence files

- `cold-start-evidence/orca-checkpoint-inspection.json`: full checkpoint integrity results, account counts, ELF hash and sampled account hashes.
- `cold-start-evidence/orca-archive-manifest.csv`: exact candidate input URLs, compressed sizes and verification levels.
- `cold-start-evidence/old-faithful-heads.json`: observed availability and sizes for epochs 1015–1033.
- `cold-start-evidence/summary.json`: computed input totals and unresolved validation status.
- `history-event-results.json`: three selected Pump experiments and their limitations.

Sources

Primary documents and datasets were accessed September 12–13, 2026. Dynamic indexes describe their observed state, not a promise of future availability. Local measurements are distinguished from publisher statements. Source-code references describe the inspected implementation, with compatibility still subject to execution testing.

1. Orca. [Whirlpool transaction replayer](#). Repository README; inspected revision 95b595cad50f1f5147c3524613c805c516f03005.
2. Orca-linked Pleiades archive. [Public file index](#). Index timestamp September 12, 2026, 12:46:03 UTC. Captured totals and index hash in `cold-start-evidence/summary.json`.
3. Pleiades archive. [August 11 checkpoint](#) and [August 12 instructions](#). Local full-checkpoint and partial-instruction inspection in `cold-start-evidence/orca-checkpoint-inspection.json`.
4. Solana public mainnet RPC. [getEpochInfo specification](#) and [getEpochSchedule specification](#). Observed response saved in `cold-start-evidence/solana-epoch-context.json`.
5. Solana. [getTransaction](#) and [RPC JSON structures](#). Transaction response and metadata definitions.
6. Solana. [Accounts](#). Account fields and ownership rules.
7. Orca. [Replayer schema](#). Checkpoint, transaction and token-file schemas.
8. Orca. [Replayer implementation](#). Daily loading, callbacks, program updates and target bounds.
9. Orca. [Swap replay adapter](#). Synthetic token account construction and oracle handling.
10. Old Faithful. [OF1 files](#). Published epoch file and index naming.
11. Old Faithful. [HTTP configuration](#). HTTP range retrieval configuration.
12. Old Faithful. [Indexes](#). Slot, transaction and address lookup roles.
13. Old Faithful. Live HEAD observations for [epoch 1015](#), [epoch 1032](#) and all intervening epochs; [epoch 1033](#) returned 404. Full URL list and byte counts in `cold-start-evidence/old-faithful-heads.json`.
14. Anza. [Jetstreamer](#). Historical streaming and indexing implementation and advertised ingestion performance.
15. Solana RPC / Triton. [Superbank](#). Historical ledger RPC stack.
16. SVMscope local experiment. `docs/research/history-event-results.json` and `docs/research/history-event-probe.md`. Selected historical transaction results; scope and limitations recorded with the measurements.
17. Pump. [Public program documentation](#) and [Pump IDL](#).
18. Solana Labs. [Upgradeable loader instruction format](#). Archived historical implementation; offset/payload writes and deployment relationships.
19. Solana Labs. [Solana Bigtable / warehouse documentation](#). Historical ledger warehouse context. Anonymous bucket-list access observations concern access only; a required-date whole-chain snapshot was not obtained.
20. Google Cloud. [Requester Pays](#). Billing-project requirements, access conditions and current promotional transfer note.
21. Anza. [Geyser plugins](#). Account-update and transaction interfaces.
22. Solana Labs. [AccountsDB PostgreSQL plugin](#). Historical account persistence support.

23. Snapshot ETL maintainers. [solana-snapshot-etl](#). Archived repository documenting streamed account extraction; contemporary format compatibility is unverified.
24. Firedancer. [Offline backtest command](#). Source inspected; no end-to-end benchmark for this import was run.
25. Overclock. [Mithril](#). Current implementation, operational notes and historical replay roadmap.
26. Solarchive. [Project site](#) and [Hugging Face dataset](#). Publisher descriptions.
27. Solarchive. [Dataset index](#) and [account-directory API listing](#). Observed latest account partition December 2025; index captured in evidence.
28. Solana RPC. [Cloudbreak](#). Snapshot and live-account serving infrastructure.
29. Solana Vibe Station. [Rewind](#). Publisher's historical state and simulation offering; access and required-date coverage not validated.
30. k256. [Introducing Replay](#). May 21, 2026. Historical frame and local replay workflow; seed availability not validated.
31. datastore.sh. [Historical dataset catalog](#). Inspected catalog, not a validated complete-state source.
32. GitHub. [svmscope-records repository API](#). Anonymous response reported public visibility.
33. GitHub. [About releases](#). Release asset limits and documented storage/bandwidth treatment.
34. GitHub. [Hosted runners reference](#) and [Actions billing](#). Public standard runner resources and pricing.
35. GitHub. [Actions limits](#). Hosted job duration limits and operational limits.
36. SVMscope working tree. `src/records.rs`, `src/records_github.rs`, `src/scope.rs`, `src/replay.rs`. Local code inspection; uncommitted changes exist and were preserved. Proposed work packages are separate from completed work.